

გაკვეთილი 3 Mod 7 (7.1)

გაკვეთილი 3 — GitHub და Remote Repository

თემა: GitHub-ის გამოყენება, Remote Repository-ის შექმნა და ლოკალური პროექტის GitHub-ზე ატვირთვა

ხანგრძლივობა: 40 წუთი

წინა გაკვეთილებიდან მოსწავლეებმა უკვე იციან: `git init`, `git status`, `git add`, `git commit`, `git log`, Repository, Untracked/Modified/Staged.

გაკვეთილის მიზანი: მოსწავლემ გაიგოს განსხვავება **Local Repository**-სა და **Remote Repository**-ს შორის, შექმნას Repository GitHub-ზე, დააკავშიროს იგი თავის ლოკალურ პროექტთან და `git push`-ის საშუალებით ატვირთოს Commit-ები GitHub-ზე.

1. წინა გაკვეთილის მოკლე გამეორება

წინა გაკვეთილზე პროექტის ცვლილებები Git-ში შევინახეთ.

მაგალითად:

```
git init
git add .
git commit -m "Create homepage"
```

შემდეგ კოდი შევცვალეთ და მეორე Commit გავაკეთეთ:

```
git add .
git commit -m "Add welcome text"
```

ისტორია შეგვეძლო გვეჩვენა:

```
git log --oneline
```

და მიგვეღო:

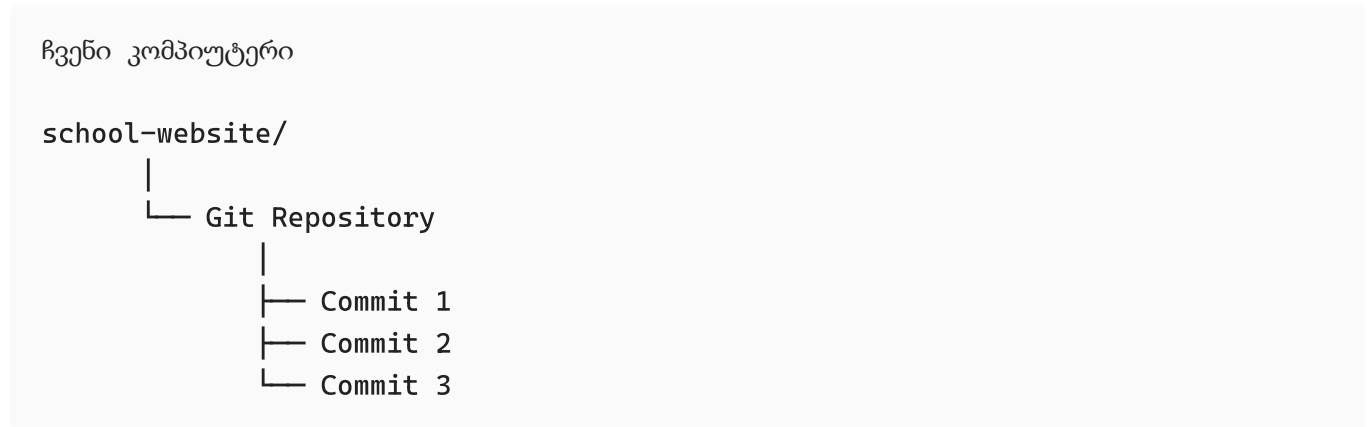
```
a72f51c Add welcome text
f812bc2 Create homepage
```

მაგრამ მნიშვნელოვანი კითხვაა:

სად ინახება ეს ყველაფერი?

ამ ეტაპზე ყველაფერი ინახება ჩვენს კომპიუტერში.

ანუ:



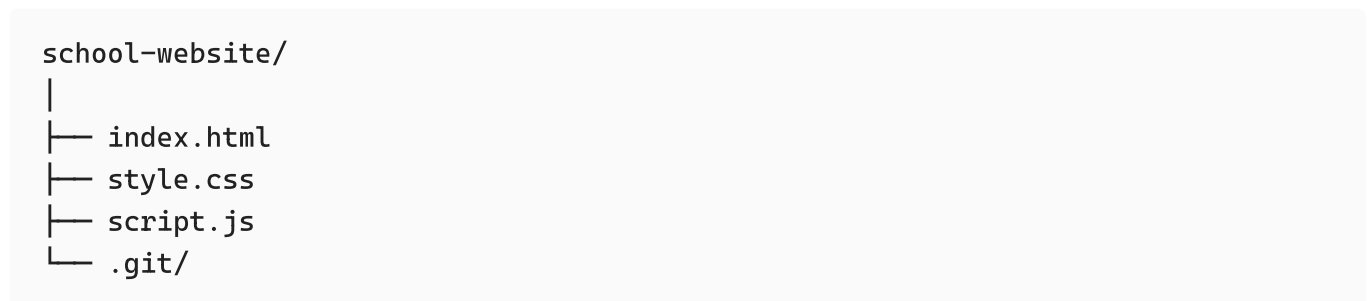
ეს არის **Local Repository**.

დღეს ვისწავლით, როგორ გადავიტანოთ ეს Repository ინტერნეტში — GitHub-ზე.

2. რა არის Local Repository?

Local Repository არის Git Repository, რომელიც ჩვენს კომპიუტერში მდებარეობს.

მაგალითად გვაქვს:



როდესაც შევასრულეთ:

```
git init
```

ჩვენს კომპიუტერში შევქმენით **Local Git Repository**.

როდესაც ვაკეთებთ:

```
git add .  
git commit -m "Create homepage"
```

Commit-იც ჯერჯერობით მხოლოდ ჩვენს კომპიუტერში ინახება.

3. რა არის Remote Repository?

Remote Repository არის Repository, რომელიც სხვა სერვერზე/ინტერნეტშია განთავსებული და არა მხოლოდ ჩვენს კომპიუტერში.

ჩვენ შემთხვევაში ამისთვის გამოვიყენებთ [GitHub](#)-ს.

მარტივად:

```
LOCAL REPOSITORY  
ჩვენს კომპიუტერში  
  
↓  
  
REMOTE REPOSITORY  
GitHub-ზე
```

მაგალითად, ჩვენი პროექტი კომპიუტერშია:

```
school-website/
```

ხოლო მისი Remote Repository შეიძლება GitHub-ზე გვქონდეს:

```
school-website
```

4. რატომ გვჭირდება GitHub?

Git უკვე გვამღევს ვერსიების კონტროლის საშუალებას.

მაშინ GitHub საერთოდ რატომ გვჭირდება?

GitHub-ის საშუალებით შეგვიძლია:

- პროექტი ონლაინ შევინახოთ;

- პროექტი სხვა კომპიუტერიდანაც მივიღოთ;
- სხვებს გავუზიაროთ ჩვენი კოდი;
- გუნდურად ვიმუშაოთ;
- ჩვენი პროექტები/პორტფოლიო სხვებს ვაჩვენოთ;
- Local Repository და Remote Repository ერთმანეთთან დავაკავშიროთ.

ამიტომ კიდევ ერთხელ:

Git — მართავს პროექტის ვერსიებს.

GitHub — გვამღევს საშუალებას Git Repository ონლაინ შევინახოთ და სხვებს გავუზიაროთ.

5. Local და Remote Repository

ეს განსხვავება აუცილებლად კარგად უნდა გავიგოთ.

Local Repository	Remote Repository
ჩვენს კომპიუტერშია	ინტერნეტში/სერვერზეა
Git-ით ვმართავთ	მაგალითად GitHub-ზე ინახება
<code>commit</code> აქ ინახავს ცვლილებას	<code>push</code> აქ აგზავნის Commit-ებს
ინტერნეტი აუცილებელი არაა	ინტერნეტი საჭიროა

ძალიან მნიშვნელოვანი განსხვავებაა:

```
git commit
```

და:

```
git push
```

`commit` და `push` ერთი და იგივე არ არის.

6. `commit` და `push` — განსხვავება

ვთქვათ, კოდში ცვლილება გავაკეთეთ.

შემდეგ:

```
git add .  
git commit -m "Add contact page"
```

რა მოხდა?

ცვლილება შევიწახეთ **Local Repository**-ში.

GitHub-ზე ჯერ არაფერი გაგვიგზავნია.

შემდეგ:

```
git push
```

Commit-ები იგზავნება **Remote Repository**-ში.

სქემა:

```
Working Directory  
  ↓  
git add  
  ↓  
Staging Area  
  ↓  
git commit  
  ↓  
Local Repository  
  ↓  
git push  
  ↓  
Remote Repository  
  (GitHub)
```

ეს არის დღევანდელი გაკვეთილის ყველაზე მნიშვნელოვანი სქემა.

7. GitHub ანგარიშის შექმნა

თუ მოსწავლეს ანგარიში არ აქვს, უნდა შევიდეს [GitHub-ის ოფიციალურ საიტზე](#) და შექმნას ანგარიში.

ამის შემდეგ GitHub-ზე შეგვიძლია Repository-ის შექმნა.

8. GitHub-ზე ახალი Repository-ის შექმნა

GitHub-ზე:

1. გაჭერთ **New repository**.

2. **Repository name** ველში ჩავწერთ, მაგალითად:

school-website

3. შეგვიძლია ავირჩიოთ:

Public

ან:

Private

Public

Repository-ის ნახვა სხვებსაც შეუძლიათ.

Private

Repository-ის ნახვა მხოლოდ ჩვენ და ის ადამიანები შეძლებენ, ვისაც წვდომას მივცემთ.

სასწავლო პროექტისთვის შეგვიძლია გამოვიყენოთ:

Public

შემდეგ ვაჭერთ:

Create repository

9. მნიშვნელოვანი დეტალი Repository-ის შექმნისას

რადგან ჩვენ უკვე გვაქვს ლოკალური პროექტი და **Commit**-ები, GitHub-ზე ახალი Repository-ის შექმნისას ჯობს თავიდან არ დავამატოთ:

README
.gitignore
License

ანუ შევქმნათ **ცარიელი Remote Repository**.

ეს პირველ დაკავშირებას უფრო მარტივს გახდის.

10. ახლა გვაქვს ორი Repository

ამ მომენტში გვაქვს:

კომპიუტერში:

```
school-website/  
|  
├─ index.html  
├─ style.css  
└─ .git/
```

ეს არის:

Local Repository

და GitHub-ზე გვაქვს:

```
school-website
```

ეს არის:

Remote Repository

მაგრამ ჯერ ეს ორი ერთმანეთთან **დაკავშირებული არ არის**.

11. Remote — რას ნიშნავს?

Git-ში **remote** ნიშნავს სხვა ადგილას მდებარე Repository-ს, რომელთანაც ჩვენი Local Repository არის დაკავშირებული.

ჩვენ შემთხვევაში Remote Repository GitHub-ზე მდებარეობს.

სქემა:

```
Local Repository
  ↓
Remote
  ↓
GitHub Repository
```

12. Repository-ის მისამართი

GitHub-ზე Repository-ის შექმნის შემდეგ მას აქვს საკუთარი მისამართი.

მაგალითად:

```
https://github.com/username/school-website.git
```

`username` -ის ადგილას იქნება კონკრეტული GitHub მომხმარებლის სახელი.

ეს მისამართი Git-ს ეუბნება:

„აი, სად მდებარეობს ჩემი Remote Repository.“

13. `git remote add origin`

Local Repository-ის GitHub Repository-სთან დასაკავშირებლად გამოიყენება:

```
git remote add origin https://github.com/username/school-website.git
```

დავშალოთ ნაწილებად:

```
git
```

ვიყენებთ Git-ს.

```
remote
```

ვმუშაობთ Remote Repository-სთან.

```
add
```

ვამატებთ ახალ Remote-ს.

```
origin
```

Remote Repository-ს ვაძლევთ სახელს.

შემდეგ:

```
https://github.com/username/school-website.git
```

არის Remote Repository-ის მისამართი.

14. რა არის `origin` ?

ეს ნაწილი თავიდან შეიძლება ცოტა დამაბნეველი იყოს.

```
git remote add origin ...
```

origin უბრალოდ სახელია, რომელსაც ჩვეულებრივ მთავარ Remote Repository-ს ვაძლევთ.

შეგვიძლია ასე წარმოვიდგინოთ:

```
origin  
↓  
https://github.com/username/school-website.git
```

ანუ იმის ნაცვლად, რომ ყოველ ჯერზე სრული მისამართი გამოვიყენოთ, Git მას `origin` სახელით იცნობს.

`origin` არ ნიშნავს GitHub-ს. ის უბრალოდ Remote-ის გავრცელებული სახელია.

15. როგორ შევამოწმოთ Remote?

დაკავშირების შემდეგ შეგვიძლია გამოვიყენოთ:

```
git remote -v
```

მაგალითად მივიღებთ:

```
origin https://github.com/username/school-website.git (fetch)
origin https://github.com/username/school-website.git (push)
```

ეს ნიშნავს, რომ Local Repository-ს უკვე აქვს Remote სახელად:

```
origin
```

16. რა არის Branch?

Branch-ებს შემდეგ გაკვეთილზე დეტალურად ვისწავლით, მაგრამ `push` -ისთვის ერთი რამ ახლა უნდა ვიცოდეთ.

Git Repository-ში ძირითადი branch ჩვეულებრივ არის:

```
main
```

შეგვიძლია წარმოვიდგინოთ:

```
Repository
|
main
|
├─ Commit 1
├─ Commit 2
└─ Commit 3
```

ამ ეტაპზე საკმარისია ვიცოდეთ:

`main` არის ჩვენი პროექტის ძირითადი branch-ის გავრცელებული სახელი.

Branch-ის იდეას მეოთხე გაკვეთილზე უფრო კარგად განვიხილავთ.

17. `git branch -M main`

იმისთვის, რომ მიმდინარე ძირითადი branch-ის სახელი `main` იყოს, შეგვიძლია გამოვიყენოთ:

```
git branch -M main
```

ამ ბრძანების შემდეგ მიმდინარე branch-ის სახელი იქნება:

```
main
```

ამ ეტაპზე მოსწავლეს არ სჭირდება `-M` პარამეტრის ყველა დეტალის ცოდნა. მთავარია იცოდეს, რომ ამ ბრძანებით მიმდინარე branch-ს `main` სახელზე ვაყენებთ/ვუცვლით სახელს.

18. პირველი `push`

ახლა ყველაფერი მზადაა.

გვაქვს:

```
Local Repository
  ↓
origin
  ↓
GitHub Repository
```

Commit-ების GitHub-ზე გასაგზავნად:

```
git push -u origin main
```

დავშალოთ:

```
git push
```

Local Repository-ში არსებული Commit-ების Remote Repository-ში გაგზავნა.

```
origin
```

რომელ Remote Repository-ში ვაგზავნით.

```
main
```

რომელი branch-ის Commit-ებს ვაგზავნით.

ამიტომ:

```
git push -u origin main
```

მარტივად ნიშნავს:

| ჩემი main branch-ის Commit-ები გააგზავნე origin Remote Repository-ში.

-u კი ამ Local და Remote branch-ს ერთმანეთთან აკავშირებს, რათა შემდეგში ხშირად უბრალოდ git push იყოს საკმარისი.

19. სრული პირველი ატვირთვა

ვთქვათ უკვე გვაქვს პროექტი და Commit:

```
git log --oneline
```

გვიჩვენებს:

```
a72f51c Add website styles  
c91ab21 Add welcome text  
f812bc2 Create homepage
```

GitHub-ზე შევქმენით ცარიელი Repository.

შემდეგ Terminal-ში:

```
git remote add origin https://github.com/username/school-website.git
```

შემდეგ:

```
git branch -M main
```

და:

```
git push -u origin main
```

ახლა GitHub-ის გვერდს თუ განვაახლებთ, უნდა გამოჩნდეს ჩვენი ფაილები:

```
index.html
style.css
```

ასევე GitHub-ზე უკვე იქნება Commit-ების ისტორიაც.

20. რა გაიგზავნა GitHub-ზე?

მნიშვნელოვანია: `push` მხოლოდ ერთი HTML ფაილის „ატვირთვა“ არ არის.

GitHub-ზე გადავიდა ჩვენი **Git-ის ისტორია** და **Commit-ები**.

ანუ თუ ლოკალურად გვაქვს:

```
Commit 1 → Create homepage
Commit 2 → Add welcome text
Commit 3 → Add website styles
```

`push` -ის შემდეგ ეს ისტორია Remote Repository-შიც გვექნება.

LOCAL		GITHUB
Commit 1	→	Commit 1
Commit 2	→	Commit 2
Commit 3	→	Commit 3

21. შემდეგი ცვლილების GitHub-ზე გაგზავნა

პირველი `push` უკვე გავაკეთეთ.

ახლა `index.html` -ში დავამატოთ:

```
<h2>About Our School</h2>
<p>Welcome to our school!</p>
```

GitHub-ზე ეს ცვლილება ავტომატურად არ გამოჩნდება.

ჯერ უნდა შევინახოთ Git-ში.

1. გამოწმეობთ:

```
git status
```

2. გამზადებით:

```
git add .
```

3. ვინახავთ Local Repository-ში:

```
git commit -m "Add about section"
```

4. ვაგზავნით GitHub-ზე:

```
git push
```

ახლა GitHub-ის გვერდს რომ განვაახლებთ, ახალი ცვლილებაც გამოჩნდება.

22. რატომ აღარ დავწერეთ სრული ბრძანება?

პირველად გამოვიყენეთ:

```
git push -u origin main
```

ამის შემდეგ ხშირად შეგვიძლია უბრალოდ:

```
git push
```

რადგან Git-ს უკვე ახსოვს, რომ ჩვენი მიმდინარე `main` დაკავშირებულია შესაბამის Remote branch-თან.

ანუ:

პირველად:

```
git push -u origin main
```

შემდეგ:

```
git push
```

23. სრული Git Workflow

ახლა უკვე პირველი სამი გაკვეთილის ცოდნა შეგვიძლია ერთ სქემაში გავაერთიანოთ.

პროექტის შექმნისას ერთხელ:

```
git init
```

მუშაობის დროს:

```
git status  
git add .  
git commit -m "Add new feature"
```

ეს ცვლილებას ინახავს **Local Repository**-ში.

შემდეგ:

```
git push
```

ეს Commit-ს აგზავნის **Remote Repository**-ში.

სრული სქემა:

```
ვცვლით კოდს  
↓  
git status  
↓  
git add .  
↓  
Staging Area  
↓  
git commit -m "message"  
↓  
Local Repository  
↓
```

```
git push
↓
GitHub
Remote Repository
```

24. commit თუ push ?

ეს განსხვავება განსაკუთრებით კარგად უნდა იცოდნენ.

სიტუაცია 1

დავწერეთ:

```
git add .
git commit -m "Add footer"
```

სად არის ახალი ვერსია?

პასუხი: Local Repository-ში.

სიტუაცია 2

შემდეგ გავუშვით:

```
git push
```

სად გაიგზავნა Commit?

პასუხი: Remote Repository-ში — ჩვენს შემთხვევაში GitHub-ზე.

ამიტომ:

```
commit → ინახავს ლოკალურად
push → აგზავნის Remote Repository-ში
```

25. პრაქტიკული სავარჯიშო 1 — შექმენი Remote Repository

მოსწავლემ უნდა:

1. შევიდეს GitHub-ზე;
2. შექმნას ახალი Repository;
3. დაარქვას:

```
my-first-github-project
```

4. შექმნას ცარიელი Repository;
5. იპოვოს Repository-ის URL.

შემდეგ უპასუხოს:

ეს **Repository Local** არის თუ **Remote**?

პასუხი:

Remote Repository, რადგან GitHub-ზე მდებარეობს.

26. პრაქტიკული სავარჯიშო 2 — დააკავშირე Repository-ები

მოსწავლეს უკვე აქვს Local Repository.

უნდა დააკავშიროს GitHub-ზე შექმნილ Repository-ს.

```
git remote add origin REPOSITORY_URL
```

შემდეგ შეამოწმოს:

```
git remote -v
```

და უპასუხოს:

რას ნიშნავს **origin** ?

ეს არის სახელი, რომელიც Remote Repository-ს მივუძღვით.

27. პრაქტიკული სავარჯიშო 3 — პირველი Push

მოსწავლემ შეასრულოს:

```
git branch -M main
```

შემდეგ:

```
git push -u origin main
```

ამის შემდეგ GitHub-ზე შეამოწმოს:

- ჩანს თუ არა `index.html` ;
- ჩანს თუ არა `style.css` ;
- ჩანს თუ არა Commit-ები.

28. პრაქტიკული სავარჯიშო 4 — ახალი ცვლილება

`index.html` -ში დაამატოს:

```
<footer>
  <p>My School Website</p>
</footer>
```

შემდეგ დამოუკიდებლად გააგზავნოს ცვლილება GitHub-ზე.

სწორი თანმიმდევრობა იქნება:

```
git status
git add .
git commit -m "Add footer"
git push
```

შემდეგ GitHub-ზე შეამოწმოს, გამოჩნდა თუ არა ახალი ცვლილება.

29. სავარჯიშო 5 — დაალაგე სწორი თანმიმდევრობით

მოცემულია:

```
git push
git commit -m "Add contact section"
git status
git add .
```

დაალაგეთ სწორი თანმიმდევრობით.

პასუხი:

```
git status
git add .
git commit -m "Add contact section"
git push
```

30. სავარჯიშო 6 — რომელი ბრძანება გვჭირდება?

1. მიინდა ვნახო, რა Remote Repository არის დაკავშირებული ჩემს პროექტთან.

?

პასუხი:

```
git remote -v
```

2. მიინდა Local Repository-ს Remote დავუმატო სახელად `origin`.

?

პასუხი:

```
git remote add origin REPOSITORY_URL
```

3. მიინდა მიმდინარე branch-ს `main` ერქვას.

?

პასუხი:

```
git branch -M main
```

4. მინდა პირველად გავაგზავნო main GitHub-ზე.

?

პასუხი:

```
git push -u origin main
```

5. Repository უკვე დაკავშირებულია და ახალი Commit-ის GitHub-ზე გაგზავნა მინდა.

?

პასუხი:

```
git push
```

31. სავარჯიშო 7 — იპოვე შეცდომა

მოსწავლემ შეცვალა index.html და მაშინვე გაუშვა:

```
git push
```

მაგრამ GitHub-ზე ახალი კოდი არ გამოჩნდა.

რატომ?

იმიტომ, რომ ცვლილება ჯერ Commit-ში არ იყო შენახული.

საჭიროა:

```
git add .  
git commit -m "Update homepage"  
git push
```

ეს ძალიან მნიშვნელოვანი პრინციპია:

`push` აგზავნის Commit-ებს. მხოლოდ ფაილის შეცვლა საკმარისი არ არის.

32. საბოლოო პრაქტიკული დავალება

მოსწავლემ გამოიყენოს წინა გაკვეთილზე შექმნილი ვებგვერდი.

მაგალითად:

```
student-website/  
|  
├─ index.html  
└─ style.css
```

ეტაპი 1

შეამოწმოს Commit-ების ისტორია:

```
git log --oneline
```

ეტაპი 2

GitHub-ზე შექმნას:

```
student-website
```

Remote Repository.

ეტაპი 3

დააკავშიროს Local და Remote Repository:

```
git remote add origin REPOSITORY_URL
```

ეტაპი 4

შეამოწმოს:

```
git remote -v
```

ეტაპი 5

მიმდინარე branch გახადოს `main` :

```
git branch -M main
```

ეტაპი 6

პირველად ატვირთოს:

```
git push -u origin main
```

ეტაპი 7

`index.html` -ში დაამატოს ახალი სექცია.

მაგალითად:

```
<section>
  <h2>Contact</h2>
  <p>Contact us for more information.</p>
</section>
```

ეტაპი 8

ცვლილება გააგზავნოს GitHub-ზე:

```
git status
git add .
git commit -m "Add contact section"
git push
```

ბოლოს GitHub-ზე უნდა ჩანდეს როგორც ახალი კოდი, ასევე ახალი Commit.